

STRAATIX ENGINEERING INTERNSHIP PROJECT | AUGUST 2026

Building Talent Atlas

An evidence-grounded hybrid RAG search system for precise talent matching

[TRY THE LIVE DEMO](#)

[Read the engineering journal](#)



Executive thesis

Talent search fails when meaning is treated as a substitute for facts. Talent Atlas combines semantic retrieval over resume evidence with exact keywords, canonical skills and structured PostgreSQL filters. The system returns one candidate-level ranking, the passages that support it and the score components behind the order. A typed agent makes the same engine conversational, while Pydantic validation, deterministic ranking and Langfuse traces keep model behavior inspectable.

The design boundary The LLM interprets intent and chooses tools. It does not invent candidates, calculate similarity, bypass hard filters or silently rewrite evidence.

What this prototype demonstrates

01 Hybrid retrieval Meaning, literal terms, exact skills and SQL constraints contribute independently.	02 Evidence-first ranking Chunks are grouped into people, scored and returned with supporting passages.	03 Typed agent access Natural language becomes validated JSON before the shared search endpoint runs.	04 Observable learning Traces, impressions and governed outcomes support measured iteration.
---	--	--	---

01 | BUSINESS PROBLEM, DATA AND ARCHITECTURE

Why this matters to Straatix

Straatix helps global companies build high-performing India teams where the first few hires disproportionately affect execution speed, culture and technical quality. A large candidate market does not automatically produce a precise shortlist: skills appear under different names, experience is distributed across resumes, and each additional combination of technology, seniority, location and operating context narrows the usable talent surface. India's scale therefore increases the value of better retrieval and verifiable evidence, not merely larger databases. [11]

1 India talent market Large, diverse and rapidly changing.	2 Role intent Meaning, seniority and operating context.	3 Hard constraints Skills, location, experience and evidence.	4 High-fit shortlist Small, explainable and actionable.
--	---	---	---

Talent surface: availability is not the same as fit; every explicit requirement narrows the execution-ready shortlist.

Intern contribution The work progressed from vector search to a reusable hybrid engine with structured filters, rank fusion, optional reranking, typed agent tools, memory safeguards and end-to-end traces. It is a credible foundation for workflow integration, not a claim of completed production deployment.

Meaning and exactness must coexist

A recruiter may describe distributed-systems experience without using that exact phrase, while PostgreSQL, Kubernetes, five years of experience or Bengaluru remain literal requirements. Dense-only search can return a plausible candidate who violates a must-have; keyword-only search can miss equivalent experience. The system therefore separates four kinds of evidence.

MEANING Vector Related passages despite different wording.	WORDS Lexical Rare technologies, acronyms and phrases.	FACTS Structured Skills, location, experience and status.	PROOF Evidence Candidate, document and source chunk.
--	--	---	--

Getting the data into a searchable shape

The starting corpus was a real public Kaggle recruitment dataset, not a fully synthetic set of resumes, but its rows did not match the application schema. Each row was transformed into a candidate record, a resume document and an evaluation case. Missing location and salary fields were generated deterministically for filter testing, emails were replaced with placeholders, and skill and experience fields were extracted with bounded rules. The source text remains real dataset content; generated attributes remain clearly marked demonstration data.

1 Candidate - identity + status - skills + experience - location + salary	2 Document - resume or notes - source metadata - content hash for dedup	3 Chunk - token-aware passage - 384-value embedding - candidate + document links
--	--	---

Why PostgreSQL and pgvector

One database keeps candidate relationships, hard filters, full-text search and vectors close enough to coordinate in a single request. pgvector supports exact distance as well as HNSW approximate search; HNSW's ef_search parameter exposes the expected recall-versus-latency trade-off. This avoided a second vector database and its synchronization path while leaving clear scaling options through connection pooling, caching and background workers. [2]

02 | THE SEARCH ENGINE

One endpoint, twenty deliberate steps

POST /search is the stable spine used by the search UI, benchmark scripts and recruiting agent. A request may exit early for a name lookup or filter-only query; a full candidate search compiles the input into a safe specification, retrieves a broad evidence pool and spends more computation only on the strongest candidates.

01-04 Understand - route detection - sanitize input - cache lookup - LLM or fallback plan	05-08 Constrain - validate fields - normalize aliases - confidence gate - intent routing	09-12 Retrieve - SQL eligible pool - dense HNSW - keyword / BM25 - exact-skill path	13-16 Rank - RRF fusion - candidate grouping - bounded relaxation - cross-encoder	17-20 Explain - feature score - MMR diversity - evidence explanation - trace + impressions
---	--	---	---	--

Three retrieval paths protect different failures

DENSE Semantic chunks Cosine similarity over pgvector HNSW catches paraphrases and related work.	LEXICAL Exact text PostgreSQL full-text search or optional BM25 protects uncommon tools and phrases.	SKILL Canonical facts Normalized candidate skills are scored directly rather than inferred from prose.
--	--	--

Fusion before judgment

$$\text{RRF}(d) = \text{sum over } r \quad [w_r / (60 + \text{rank}_r(d))]$$

Reciprocal Rank Fusion compares positions instead of adding incompatible cosine, text and binary scores. A candidate supported by several branches accumulates evidence. Chunks are then collapsed to one person, retaining the best passage plus supporting passages from other documents. [5]

Failure handling Empty searches receive at most one recorded relaxation. Status is never silently relaxed, and every broadened condition is returned to the UI.

03 | MODELS, RANKING AND EVALUATION

Small models where their training objective fits

all-MiniLM-L6-v2 for retrieval

The local bi-encoder maps sentences and short paragraphs into 384-dimensional vectors. Its model card reports 22.7M parameters and contrastive training over more than one billion sentence pairs. That objective suits fast semantic recall, and the compact vectors reduce storage and memory compared with 1,536-dimensional defaults. The 256-word-piece truncation limit also reinforced token-aware chunking instead of embedding entire resumes. [3]

MS MARCO MiniLM cross-encoder for reranking

A cross-encoder reads the recruiter request and one retrieved passage together, making it a more careful relevance judge than independent embeddings. The selected L6 model was trained for passage ranking and offers a practical speed-quality point. It is applied only to a top-N prefix because joint encoding is too expensive for the full corpus. [4]

The final score uses candidate evidence, not model confidence alone

With the cross-encoder enabled, the feature score weights cross-encoder relevance at 45%, fused retrieval at 20%, exact skills at 12%, soft preferences at 10%, experience fit at 5%, skill recency at 3%, completeness at 2% and confirmed personalization at 3%. Without a cross-encoder, the missing weight moves mainly to retrieval, exact skills and experience. Scores are normalized to 0-100, then MMR selects the final list with $\lambda = 0.70$: 70% relevance and 30% duplicate penalty.

Recorded strategy benchmark	Top-1	P50	P95	Interpretation
Semantic only	35.0%	53 ms	172 ms	fast baseline
RRF hybrid	35.0%	60 ms	85 ms	stable hybrid
RRF + rerank	28.75%	244 ms	450 ms	optional

Benchmark context. 80 queries (30 manually designed and 50 imported), top_k=10, rerank pool=30, recorded 15 May 2026. The planner fell back on all 80 runs, labels were limited and results should be read as project evidence rather than a universal model comparison.

Decision Reranking stayed configurable. A stronger theoretical model was not allowed to override a negative project result without a better labeled evaluation set.

04 | AGENT, INTERFACE AND OBSERVABILITY

The chatbot calls the search engine; it does not replace it

JSON has two related jobs. A direct POST /search body configures predefined pipeline stages; it cannot execute arbitrary code. In the chatbot, PydanticAI exposes a fixed registry of typed Python functions. The model chooses one function and emits JSON arguments for its generated schema; Pydantic validates them before the corresponding `do_*` helper runs. Search tools call the same engine, inspection tools read candidate evidence, and workflow tools update explicit session state. [6]

1 Recruiter message Plain language plus current session context.	2 Fixed tool registry Choose search, inspect, refine or workflow function.	3 Validated JSON args Match the selected function's generated schema.	4 Deterministic helper Run shared search, database or session logic.	5 JSON to the UI Translate tool results into SSE status, cards and prose.
--	--	---	--	---

Prompting as an operating policy

The prompt teaches intent-to-function routing as well as argument formatting: a new brief uses `run_search`; a small refinement uses `modify_and_search`; exact vocabulary can use `list_skills` or `keyword_search`; comparisons fetch candidate details; and shortlist actions use explicit workflow tools. Search results return JSON containing ordered candidates, evidence, scores and diagnostics. The SSE translator observes each tool start, arguments and result, renders status and candidate cards, then lets the model summarize only that returned evidence. Validation still happens in code after generation.

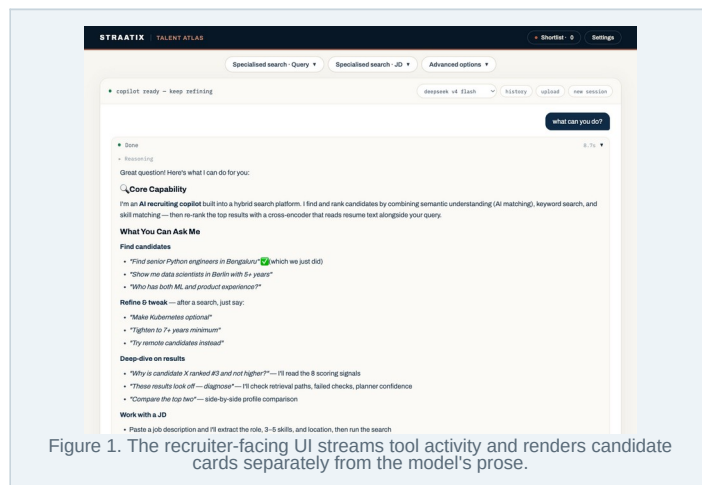
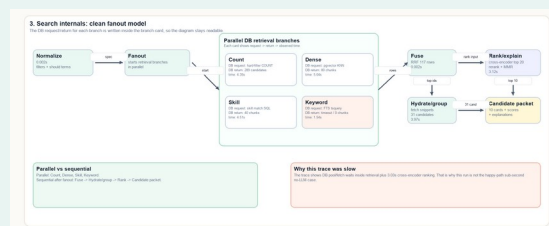


Figure 1. The recruiter-facing UI streams tool activity and renders candidate cards separately from the model's prose.



From 'slow' to a traceable cause

Langfuse nests the request, model generation, tool calls, SQL, retrieval branches, fusion and reranking. That separates model latency from database waits and ranking cost, supports replay and allows production content to be sampled and redacted. [7]

Figure 2. Search spans from a recorded trace.

Learning without silent bias

Search impressions and explicit outcomes create a future learning dataset, but unconfirmed observations cannot affect ranking. Confirmed preferences are stored separately, personalization can be disabled, and every score remains decomposable into named signals.

05 | VALUE, LIMITS AND NEXT STEPS

A credible foundation, with clear claims still to earn

SPEED One reusable engine The UI, benchmarks and agent share ranking logic instead of maintaining divergent search paths.	TRUST Auditable evidence Each result carries matching passages, normalized intent, component scores and trace metadata.	LEARNING Measured iteration Outcomes and traces can support tuning by role family without treating the LLM as an oracle.
---	---	--

What is proven - and what is not

Demonstrated in the prototype A working hybrid retrieval architecture; transformed real Kaggle recruitment data; stable typed search and agent-tool contracts; candidate evidence and explanations; cache benchmarks; and concurrent HTTP load tests from 1 to 500 simultaneous requests, with queueing limits captured in traces.	Still requires production evidence Validation against an actual Straatix or client production dataset and real recruiter traffic; a large recruiter-adjudicated benchmark; measured business uplift; tenant isolation; PII governance; fairness analysis; and workflow integration.
--	---

A pragmatic 90-day path

0-30 DAYS Production-data validation - sample real role + profile data - 100-200 recruiter judgments - Recall@50 and nDCG@10	31-60 DAYS Harden - privacy + tenant controls - migration rehearsals - cancellation + queue safeguards	61-90 DAYS Integrate + learn - controlled recruiter pilot - explicit rejection reasons - role-family drift review
---	---	--

References and project evidence

[1] Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020. [Source](#)

[2] pgvector documentation: vector search, HNSW, filtering and hybrid retrieval. [Source](#)

[3] Sentence Transformers model card: all-MiniLM-L6-v2. [Source](#)

[4] Cross-Encoder model card: ms-marco-MiniLM-L6-v2. [Source](#)

[5] Cormack, Clarke and Buettcher, Reciprocal Rank Fusion, SIGIR 2009. [Source](#)

[6] PydanticAI model and provider abstraction. [Source](#)

[7] Langfuse observability and application tracing. [Source](#)

[8] Talent Atlas engineering journal. [Source](#)

[9] Talent Atlas source repository and recorded benchmark artifacts. [Source](#)

[10] Interactive Talent Atlas demonstration. [Source](#)

[11] Government of India, Press Information Bureau: India Leads Globally in AI Talent Acquisition, 19 Dec 2025. [Source](#)

Prepared as a concise engineering whitepaper for the Straatix Talent Atlas prototype. Forward-looking recommendations should be validated through labeled evaluation, privacy review and production testing.